

Procesamiento Masivo de Datos**Profesor:** Alex Di Genova**Integrantes:** Juan I. Riquelme y Martín Azua

Tarea 1

P1. (Procesamiento de datos con comandos Linux.) Resuelva los siguientes problemas utilizando comandos Linux. En los apartados que lo indiquen, la transformación principal debe realizarse con **awk**.

1. Utilizando el archivo `worldcitiespop.csv.gz`, calcule el promedio de habitantes por ciudad y país.

```
1 zcat worldcitiespop.csv.gz | awk -F',' '
2 NR > 1 && $5 != "" {
3     lugar = $1 "," $2
4     suma[lugar] += $5
5     cantidad[lugar]++
6 }
7
8 END {
9     print "Pais,Ciudad,Promedio_habitantes"
10
11     for (lugar in suma) {
12         print lugar "," suma[lugar] / cantidad[lugar]
13     }
14 }
15 '
```

Nota: Para usar lo anterior, el archivo `worldcitiespop.csv.gz` tiene que estar descomprimido.

2. Utilizando **awk**, determine las 10 ciudades con mayor población.

```
1 zcat worldcitiespop.csv.gz |
2 awk -F',' 'NR > 1 && $5 != "" {print $5, $3, $1}' |
3 sort -nr |
4 head -n 10
```

3. Utilizando **awk**, determine el porcentaje de ciudades sin población de cada país.

```
1 zcat worldcitiespop.csv.gz |
2 awk -F',' '
3 NR > 1 {
4     total[$1]++
5     if ($5 == "") vacias[$1]++
6 }
7 END {
8     for (pais in total)
9         print pais, 100 * vacias[pais] / total[pais] "%"
10 }'
```

4. Calcule el número de habitantes y la ciudad más poblada de América del Sur.

```
1 zcat worldcitiespop.csv.gz |
2 awk -F',' '
3 NR > 1 &&
4 tolower($1) ~ /^(ar|bo|br|cl|co|ec|fk|gf|gy|pe|py|sr|uy|ve)$/ &&
5 $5 > maximo {
6     maximo = $5
7     ciudad = $3
8     pais = $1
9 }
10 END {
11     print ciudad, pais, maximo
12 }'
```

5. Determine el máximo y el mínimo de cada columna del archivo `matrix.txt`.

```
1 awk '
2 {
3     for (i = 1; i <= NF; i++) {
4         if (NR == 1 || $i < minimo[i])
5             minimo[i] = $i
6
7         if (NR == 1 || $i > maximo[i])
8             maximo[i] = $i
9     }
10 }
11
12 END {
13     for (i = 1; i <= NF; i++)
14         print i, minimo[i], maximo[i]
15 }
16 ' matrix.txt
```

6. Invierta el orden de las columnas de `matrix.txt` (50,49,...,1).

```
1 awk '
2 {
3     for (i = NF; i > 1; i--)
4         printf $i
5
6     print $1
7 }
8 ' matrix.txt
```

7. Seleccione las filas de `matrix.txt` cuyo promedio sea mayor que el promedio general del archivo.

```
1 awk '
2 FNR == NR {
3     for (i = 1; i <= NF; i++) {
4         total += $i
5         cantidad++
6     } next
7 }
8 {suma = 0
9     for (i = 1; i <= NF; i++)
10         suma += $i
11
12     if (suma / NF > total / cantidad)
13         print
14 }
15 ' matrix.txt
```

8. Determine cuántos números impares contiene `matrix.txt`.

```
1 awk '
2 {for (i = 1; i <= NF; i++) {
3     if ($i % 2 != 0)
4         impares++
5     }}
6 END {
7     print "Cantidad de numeros impares:", impares
8 }' matrix.txt
```

9. Determine cuántos números son divisibles por 4 y por 9 en `matrix.txt`. Reporte claramente si se cuentan ambas condiciones por separado y, además, cuántos cumplen ambas simultáneamente.

```
1 awk '
2 {for (i = 1; i <= NF; i++) {
3     if ($i % 4 == 0)
4         divisibles_4++
5     if ($i % 9 == 0)
6         divisibles_9++
7     if ($i % 4 == 0 && $i % 9 == 0)
8         divisibles_ambos++}}
9 END {
10     print "Divisibles por 4:", divisibles_4
11     print "Divisibles por 9:", divisibles_9
12     print "Divisibles por 4 y por 9:", divisibles_ambos
13 }
14 ' matrix.txt
```

10. Imprima la fila de `matrix.txt` con la mayor suma acumulada.

```

1  awk '
2  { suma_filas = 0
3    for (i = 1; i <= NF; i++)
4      suma_filas += $i
5    if (NR == 1 || suma_filas > suma_maxima) {
6      suma_maxima = suma_filas
7      numero_filas = NR
8      filas_maximas = $0
9    }
10 END {
11   print "Numero de filas:", numero_filas
12   print "Suma maxima:", suma_maxima
13   print "Contenido de la fila:", filas_maximas
14 }
15 ' matrix.txt

```

P2. Se debe estimar una posición de una lectura mediante coincidencias probabilísticas de semillas, sin programación dinámica, distancia de edición, verificación base a base ni un alineador externo. Se entregan AWK secuencial, C secuencial y C con POSIX Threads.

Las decisiones que fijan una única definición del algoritmo son:

1. Se usa únicamente la hebra directa. No se calcula complemento reverso.
2. Se implementa la versión básica, sin firma de 16 bits. Las colisiones no se detectan ni se resuelven.
3. Cada celda conserva la **primera** posición observada y el identificador de su secuencia; todas las inserciones incrementan el contador.
4. Por enunciado, se usa el mismo salto L al construir y consultar el índice. El valor inicial del límite de copias es $c = 1$; también se experimenta con $c = 2$.
5. Se aceptan mayúsculas y minúsculas. Por enunciado, se omiten semillas con caracteres distintos de A, C, G y T. Las bases ambiguas sí cuentan en G y en las coordenadas; una semilla no atraviesa el límite entre secuencias FASTA.
6. Se descarta un candidato cuyo intervalo quedaría fuera del contig. Esta decisión adicional es un filtro geométrico y es común a las tres versiones.
7. Los identificadores de salida son el primer campo del encabezado. Los lectores verifican que cada calidad tenga el mismo largo que la secuencia. Las calidades no intervienen en el puntaje.

1. Construcción del índice Sea $G = \sum_{r=1}^R g_r$ el largo total, incluyendo bases ambiguas. Para $d \in \{1, 2, 4, 8\}$ se define

$$M = \left\lfloor \frac{G}{d} \right\rfloor.$$

Por enunciado, estos cuatro tamaños deben compararse. El programa admite además un M explícito. Se exige $1 \leq M \leq \min(G, 4294967291)$; si un ejemplo diminuto produce $M = 0$, el programa solicita cambiar los parámetros.

La primera pasada del FASTA cuenta largos y guarda nombres. La segunda inserta las semillas que comienzan en $1, 1+L, 1+2L, \dots$, siempre que quepan completas. La referencia no se conserva completa en RAM. En C se mantiene una ventana circular duplicada de hasta 62 caracteres. En AWK se conservan hasta $k-1$ bases de la línea anterior para unir semillas que cruzan saltos de línea.

2. Función de hash y equivalencia numérica Definimos

$$b(A) = 1, \quad b(C) = 2, \quad b(G) = 3, \quad b(T) = 4, \quad B = 131, \quad P = 4294967291.$$

Para una semilla $s_1 \cdots s_k$, se empieza con $h_0 = 0$ y se calcula

$$h_j = (131h_{j-1} + b(s_j)) \text{ mód } P, \quad j = 1, \dots, k.$$

Finalmente, el índice de la celda, contado desde cero, es $i = h_k \text{ mód } M$. Equivalentemente,

$$h_k = \left(\sum_{j=1}^k b(s_j) 131^{k-j} \right) \text{ mód } P.$$

El orden de las bases cambia el polinomio. Reducir módulo P mantiene el estado acotado y evita desbordamiento al procesar $k = 31$. Los códigos no nulos evitan que una A inicial actúe como un dígito nulo. Este hash es determinista y sencillo de reproducir; no ofrece una garantía criptográfica ni elimina colisiones.

Por qué C y AWK calculan lo mismo. En cada paso $0 \leq h_j < P$, luego

$$131h_{j-1} + b(s_j) \leq 131(P-1) + 4 = 562640714994 < 2^{40} < 2^{53}.$$

Los enteros intermedios son representables exactamente en un `double` binario de 53 bits de precisión, como el usado por AWK en la prueba. En C se emplea `uint64_t`. Así no se necesita representar el entero de 62 bits que resultaría de codificar directamente las 31 bases usando dos bits por base. Se fija también el rango común de coordenadas y parámetros en $2^{53} - 1$.

Contenido de una celda en C.

Campo	Tipo	Significado
position	<code>uint64_t</code>	Primera posición, basada en 1
count	<code>uint32_t</code>	Número de inserciones
reference	<code>uint32_t</code>	Orden del contig en FASTA

La plataforma de prueba usa 16 bytes por celda. Se detecta un eventual desbordamiento del contador de 32 bits y se termina con un error explícito. AWK usa arreglos asociativos para representar las M celdas lógicas; una celda ausente tiene contador cero. Por ello su memoria no es simplemente $16M$.

Si una lectura tiene largo n , se consultan las semillas en desplazamientos $j = 0, L, 2L, \dots$, con $j + k \leq n$. Por enunciado, solo se usan las celdas con contador entre 1 y c . Si la celda contiene posición p y contig r , propone

$$a = p - j.$$

Aquí p y a están basados en 1, mientras j es un desplazamiento basado en cero. Si la posición dentro de la lectura se escribiera basada en 1 como $q = j + 1$, la misma fórmula sería $a = p - q + 1$. Esto evita un error de una base.

El candidato se acepta si $a \geq 1$ y $a + n - 1 \leq g_r$. Su puntaje es

$$\text{score}(r, a) = \#\{\text{semillas consultadas que proponen exactamente } (r, a)\}.$$

Por enunciado, el orden es puntaje decreciente, posición creciente y, en caso de nuevo empate, orden creciente de las secuencias FASTA. Es importante que el contig forme parte de la clave: dos posiciones iguales en contigs distintos no deben sumar votos entre sí.

Por enunciado, una lectura sin votos es **no-hit**; una lectura con al menos un voto es **hit**. La salida incluye encabezado y una línea tabulada por lectura, en el orden de entrada:

read_id	status	reference	position	score
---------	--------	-----------	----------	-------

Para **no-hit** se imprimen *, 0, 0 en las tres últimas columnas. Se produce además un resumen con total de lecturas, hits, no-hits, histograma de puntajes, parámetros, ocupación y celdas descartadas por c .

1. Implementaciones

1.1. AWK secuencial

mapper.awk es la versión secuencial en AWK del mismo alineador por k-mers, pensada para reproducir bit a bit los resultados de las versiones en C con datasets chicos (E. coli o pruebas), aunque el propio script documenta que no escala bien al genoma humano por el overhead de los arrays asociativos de AWK; en el BEGIN inicial carga el FASTA de referencia línea por línea con getline, concatenando la secuencia de cada encabezado > en el array seq[] y calculando el largo total del genoma G (usado como valor por defecto de M si no se especifica), y luego construye el índice muestreando un k-mer cada L bases por referencia con hash_kmer, guardando en arrays paralelos (cellCount, cellPos, cellRef, cellFp, cellCol) el contador de visitas, la primera posición observada (0-based), el id de referencia, el fingerprint y el flag de colisión (activable con USE_FP=1), usando exactamente el mismo rolling hash polinomial módulo un primo de 32 bits (HASH_P, HASH_B1, HASH_B2) que las versiones en C, justificado en un extenso comentario porque el mix64 de 64 bits del enunciado pierde precisión en el tipo double de AWK; el bloque principal (regla sin patrón que se ejecuta por cada línea de stdin) procesa el FASTQ leyendo de a 4 líneas por registro (usando lineno % 4 para distinguir id/secuencia/plus/calidad), y para cada lectura muestrea sus propios k-mers, consulta el índice, acumula votos en el array votes[] por candidato (referencia, posición) usando la misma fórmula pcandidato = posición_referencia - posición_lectura, elige el candidato con más votos desempata por menor posición y luego

menor id de referencia, e imprime por stdout la línea `read_id status reference position score` (o `no-hit * 0 0`); finalmente el bloque `END` imprime por stderr un resumen con el total de lecturas, aciertos, porcentaje de hits y distribución de scores, igual que las versiones en C.

1.2. C secuencial

`mapper.c` es la versión secuencial en C del alineador probabilístico de lecturas por k-mers: lee el genoma de referencia en FASTA (`load_fasta`) y construye un índice hash (`build_index`) muestreando un k-mer cada `L` bases, donde cada celda guarda un contador de visitas, la primera posición observada, el id de referencia y, opcionalmente, un fingerprint de 16 bits para detectar colisiones; el hash usado es un rolling hash polinomial módulo un primo de 32 bits, diseñado a propósito (en vez del de 64 bits del enunciado) para poder reproducirse exacto en AWK. Luego procesa el FASTQ de lecturas registro por registro (`read_fastq_record`), y para cada lectura muestrea sus propios k-mers, consulta el índice, acumula votos por posición candidata (`pcandidato = posición_referencia - posición_lectura`) y elige el candidato con más votos, desempata por menor posición y luego menor id de referencia (`process_read + pick_best`); el resultado de cada lectura (hit/no-hit, referencia, posición, score) se imprime por stdout, mientras que por stderr se reporta un resumen con estadísticas de memoria, tiempo, ocupación del índice y distribución de scores, útil para completar las tablas del informe.

1.3. C con POSIX Threads

`mapper_pthreads.c` es la versión paralela en C del mismo alineador, y reutiliza exactamente la misma función de hash, construcción de índice, votación y desempate que `mapper.c` (por eso ambas dan resultados bit-idénticos), pero paraleliza el procesamiento de lecturas con un esquema productor-consumidor: un hilo productor (`producer_main`) lee el FASTQ secuencialmente y arma bloques de hasta `X` lecturas etiquetados con un `block_id` creciente, que va depositando en una cola acotada de capacidad `Y` (`bounded_queue`, protegida por mutex y variables de condición `not_full/not_empty`, sin espera activa); luego `N` hilos trabajadores (`worker_main`) sacan bloques de la cola en cualquier orden, consultan el índice compartido de solo lectura (sin necesidad de sincronización, porque nunca se modifica durante el mapeo) y procesan cada lectura del bloque igual que la versión secuencial. Como los bloques pueden terminar de procesarse fuera de orden, cada trabajador no imprime directamente sino que guarda su bloque resuelto en un arreglo `finished[block_id]` bajo un mutex de salida (`out_mutex`), y luego "drena" (`drain_ready_blocks`) hacia stdout todos los bloques consecutivos disponibles a partir de `next_to_print`, lo que garantiza una salida determinística e idéntica al orden de entrada sin importar cuántos hilos se usen; el main arma la referencia y el índice antes de lanzar hilos, crea el productor y los `nthreads` trabajadores, espera a que todos terminen (`pthread_join`) y finalmente reporta por stderr el mismo resumen de estadísticas que la versión secuencial (más el número de hilos usados), quedando lista para comparar tiempos y calcular aceleración/eficiencia respecto de `mapper.c`.